

OBJECT ORIENTED PROGRAMMING

UNIT - IV

**AI303PC: OBJECT ORIENTED PROGRAMMING
USING JAVA**



EVENT HANDLING

- For the user to interact with a GUI, the underlying operating system must support event handling.
 - 1) operating systems constantly monitor events such as keystrokes, mouse clicks, voice command, etc.
 - 2) operating systems sort out these events and report them to the appropriate application programs
 - 3) each application program then decides what to do in response to these events

EVENTS

- An event is an object that describes a state change in a source.
- It can be generated as a consequence of a person interacting with the elements in a graphical user interface.
- Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse.
- Events may also occur that are not directly caused by interactions with a user interface.
- For example, an event may be generated when a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed.
- Events can be defined as needed and appropriate by application.

EVENT SOURCES

- A source is an object that generates an event.
- This occurs when the internal state of that object changes in some way.
- Sources may generate more than one type of event.
- A source must register listeners in order for the listeners to receive notifications about a specific type of event.
- Each type of event has its own registration method.

EVENT SOURCES

- General form is:
 - `void addTypeListener(TypeListener el)`
- Here, Type is the name of the event and el is a reference to the event listener.
- For example,
 - 1. The method that registers a keyboard event listener is called `addKeyListener()`.
 - 2. The method that registers a mouse motion listener is called `addMouseMotionListener()`.

EVENT SOURCES

- When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as multicasting the event.
- In all cases, notifications are sent only to listeners that register to receive them.
- Some sources may allow only one listener to register. The general form is:
 - `public void addTypeListener(TypeListener el) throws java.util.TooManyListenersException`
- Here `Type` is the name of the event and `el` is a reference to the event listener.
- When such an event occurs, the registered listener is notified. This is known as unicasting the event.

EVENT SOURCES

- A source must also provide a method that allows a listener to unregister an interest in a specific type of event.
- The general form is:
 - `public void removeTypeListener(TypeListener el)`
- Here, Type is the name of the event and el is a reference to the event listener.
- For example, to remove a keyboard listener, you would call `removeKeyListener( )`.
- The methods that add or remove listeners are provided by the source that generates events.
- For example, the Component class provides methods to add and remove keyboard and mouse event listeners.

EVENT CLASSES

- The Event classes that represent events are at the core of Java's event handling mechanism.
- Super class of the Java event class hierarchy is `EventObject`, which is in `java.util.` for all events.
- Constructor is :
 - `EventObject(Object src)`
 - Here, `src` is the object that generates this event.
- `EventObject` contains two methods: `getSource()` and `toString()`.
- 1. The `getSource()` method returns the source of the event. General form is : `Object getSource()`
- 2. The `toString()` returns the string equivalent of the event.

EVENT CLASSES

- `EventObject` is a superclass of all events.
- `AWTEvent` is a superclass of all AWT events that are handled by the delegation event model.
- The package `java.awt.event` defines several types of events that are generated by various user interface elements.

EVENT CLASSES IN JAVA.AWT.EVENT

- **ActionEvent:** Generated when a button is pressed, a list item is double clicked, or a menu item is selected.
- **AdjustmentEvent:** Generated when a scroll bar is manipulated.
- **ComponentEvent:** Generated when a component is hidden, moved, resized, or becomes visible.
- **ContainerEvent:** Generated when a component is added to or removed from a container.
- **FocusEvent:** Generated when a component gains or loses keyboard focus.

EVENT CLASSES IN JAVA.AWT.EVENT

- **InputEvent:** Abstract super class for all component input event classes.
- **ItemEvent:** Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
- **KeyEvent:** Generated when input is received from the keyboard.
- **MouseEvent:** Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
- **TextEvent:** Generated when the value of a text area or text field is changed.
- **WindowEvent:** Generated when a window is activated, closed, deactivated, deiconified, iconified, opened, or quit.

EVENT LISTENERS

- A listener is an object that is notified when an event occurs.
- Event has two major requirements.
 - 1. must have been registered with one or more sources to receive notifications about specific types of events.
 - 2. It must implement methods to receive and process these notifications.
- The methods that receive and process events are defined in a set of interfaces found in `java.awt.event`.
- For example, the `MouseMotionListener` interface defines two methods to receive notifications when the mouse is dragged or moved.
- Any object may receive and process one or both of these events if it provides an implementation of this interface.

DELEGATION EVENT MODEL

- The modern approach to handling events is based on the delegation event model, which defines standard and consistent mechanisms to generate and process events.
- Its concept is quite simple: a source generates an event and sends it to one or more listeners.
- In this scheme, the listener simply waits until it receives an event.
- Once received, the listener processes the event and then returns.
- The advantage of this design is that the application logic that processes events is cleanly separated from the user interface logic that generates those events.
- A user interface element is able to "delegate" the processing of an event to a separate piece of code.

DELEGATION EVENT MODEL

- In the delegation event model, listeners must register with a source in order to receive an event notification. This provides an important benefit: notifications are sent only to listeners that want to receive them.
- This is a more efficient way to handle events than the design used by the old Java 1.0 approach. Previously, an event was propagated up the containment hierarchy until it was handled by a component.
- This required components to receive events that they did not process, and it wasted valuable time. The delegation event model eliminates this overhead.
- Note
 - Java also allows you to process events without using the delegation event model.
 - This can be done by extending an AWT component.

HANDLING MOUSE EVENTS

- Mouse events can be handled by implementing the `MouseListener` and the `MouseMotionListener` interfaces.
- `MouseListener` Interface defines five methods. The general forms of these methods are:
 - `void mouseClicked(MouseEvent me)`
 - `void mouseEntered(MouseEvent me)`
 - `void mouseExited(MouseEvent me)`
 - `void mousePressed(MouseEvent me)`
 - `void mouseReleased(MouseEvent me)`
- `MouseMotionListener` Interface. This interface defines two methods. Their general forms are :
 - `void mouseDragged(MouseEvent me)`
 - `void mouseMoved(MouseEvent me)`

HANDLING KEYBOARD EVENTS

- Keyboard events, can be handled by implementing the `KeyListener` interface.
- `KeyListener` interface defines three methods. The general forms of these methods are :
 - `void keyPressed(KeyEvent ke)`
 - `void keyReleased(KeyEvent ke)`
 - `void keyTyped(KeyEvent ke)`
- To implement keyboard events implementation to the above methods is needed.

ADAPTER CLASSES

- Java provides a special feature, called an adapter class, that can simplify the creation of event handlers.
- An adapter class provides an empty implementation of all methods in an event listener interface.
- Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface.
- You can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which you are interested.

ADAPTER CLASSES

- Adapter classes in java.awt.event are.

Adapter Class	Listener Interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener

INNER CLASSES

- Inner classes, which allow one class to be defined within another.
- An inner class is a non-static nested class. It has access to all of the variables and methods of its outer class and may refer to them directly in the same way that other non-static members of the outer class do.
- An inner class is fully within the scope of its enclosing class.
- an inner class has access to all of the members of its enclosing class, but the reverse is not true.
- Members of the inner class are known only within the scope of the inner class and may not be used by the outer class

LAYOUT MANAGER

- A layout manager automatically arranges your controls within a window by using some type of algorithm.
- it is very tedious to manually lay out a large number of components and sometimes the width and height information is not yet available when you need to arrange some control, because the native toolkit components haven't been realized.
- Each Container object has a layout manager associated with it.
- A layout manager is an instance of any class that implements the `LayoutManager` interface.
- The layout manager is set by the `setLayout( )` method. If no call to `setLayout( )` is made, then the default layout manager is used.
- Whenever a container is resized (or sized for the first time), the layout manager is used to position each of the components within it.

LAYOUT MANAGER TYPES

- Layout manager class defines the
- following types of layout managers
- Boarder Layout
- Grid Layout
- Flow Layout
- Card Layout
- GridBag Layout

BOARDER LAYOUT

- The BorderLayout class implements a common layout style for top-level windows. It has four narrow, fixed-width components at the edges and one large area in the center.
- The four sides are referred to as north, south, east, and west. The middle area is called the center.
- The constructors defined by BorderLayout:
 - BorderLayout()
 - BorderLayout(int horz, int vert)
- BorderLayout defines the following constants that specify the regions:
 - BorderLayout.CENTER | BorderLayout.SOUTH
 - BorderLayout.EAST | BorderLayout.WEST
 - BorderLayout.NORTH
- Components can be added by
 - void add(Component compObj, Object region);

GRID LAYOUT

- GridLayout lays out components in a two-dimensional grid. When you instantiate a GridLayout, you define the number of rows and columns.
- The constructors are:
 - GridLayout()
 - GridLayout(int numRows, int numColumns)
 - GridLayout(int numRows, int numColumns, int horz, int vert)
- The first form creates a single-column grid layout.
- The second form creates a grid layout with the specified number of rows and columns.
- The third form allows you to specify the horizontal and vertical space left between components in horz and vert, respectively.
- Either numRows or numColumns can be zero. Specifying numRows as zero allows for unlimited-length columns. Specifying numColumns as zero allows for unlimited-length rows.

FLOW LAYOUT

- FlowLayout is the default layout manager.
- Components are laid out from the upper-left corner, left to right and top to bottom. When no more components fit on a line, the next one appears on the next line. A small space is left between each component, above and below, as well as left and right.
- The constructors are
 - FlowLayout()
 - FlowLayout(int how)
 - FlowLayout(int how, int horz, int vert)
- The first form creates the default layout, which centers components and leaves five pixels of space between each component.

FLOW LAYOUT

- The second form allows to specify how each line is aligned. Valid values for are:
 - `FlowLayout.LEFT`
 - `FlowLayout.CENTER`
 - `FlowLayout.RIGHT`
- These values specify left, center, and right alignment, respectively.
- The third form allows to specify the horizontal and vertical space left between components in `horz` and `vert`, respectively

CARD LAYOUT

- The CardLayout class is unique among the other layout managers in that it stores several different layouts.
- Each layout can be thought of as being on a separate index card in a deck that can be shuffled so that any card is on top at a given time.
- CardLayout provides these two constructors:
 - CardLayout()
 - CardLayout(int horz, int vert)
- The cards are held in an object of type Panel. This panel must have CardLayout selected as its layout manager.
- Cards are added to panel using
 - void add(Component panelObj, Object name);
- methods defined by CardLayout:

▪ void first(Container deck)		void last(Container deck)
▪ void next(Container deck)		void previous(Container deck)
▪ void show(Container deck, String cardName)		

GRIDBAG LAYOUT

- The Grid bag layout displays components subject to the constraints specified by GridBagConstraints.
- GridLayout lays out components in a two-dimensional grid.
- The constructors are
 - GridLayout()
 - GridLayout(int numRows, int numColumns)
 - GridLayout(int numRows, int numColumns, int horz, int vert)

INTRODUCTION TO SWINGS

- Swing is a set of classes that provides more powerful and flexible components than are possible with the AWT.
- In addition to the familiar components, such as buttons, check boxes, and labels, Swing supplies several exciting additions, including tabbed panes, scroll panes, trees, and tables.
- Even familiar components such as buttons have more capabilities in Swing.

INTRODUCTION TO SWINGS

- For example, a button may have both an image and a text string associated with it. Also, the image can be changed as the state of the button changes.
- Unlike AWT components, Swing components are not implemented by platform-specific code.
- Instead, they are written entirely in Java and, therefore, are platform-independent.
- The term lightweight is used to describe such elements.

SWING COMPONENTS

- The Swing components are defined in `javax.swing`
 - `AbstractButton`: Abstract superclass for Swing buttons.
 - `ButtonGroup`: Encapsulates a mutually exclusive set of buttons.
 - `ImageIcon`: Encapsulates an icon.
 - `JApplet`: The Swing version of Applet.
 - `JButton`: The Swing push button class.
 - `JCheckBox`: The Swing check box class.
 - `JComboBox` : Encapsulates a combo box (an combination of a drop-down list and text field).

SWING COMPONENTS

- JLabel: The Swing version of a label.
- JRadioButton: The Swing version of a radio button.
- JScrollPane: Encapsulates a scrollable window.
- JTabbedPane: Encapsulates a tabbed window.
- JTable: Encapsulates a table-based control.
- JTextField: The Swing version of a text field.
- JTree: Encapsulates a tree-based control.

LIMITATIONS OF AWT

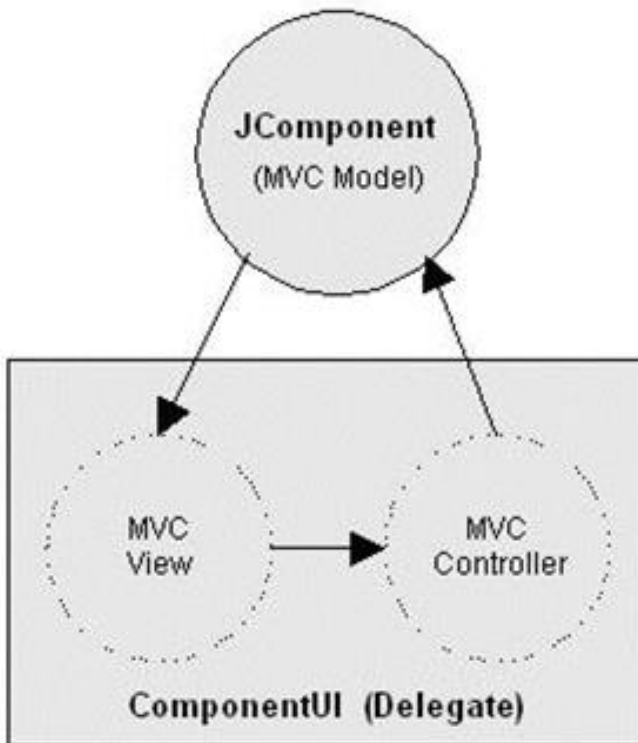
- AWT supports limited number of GUI components.
- AWT components are heavy weight components.
- AWT components are developed by using platform specific code.
- AWT components behaves differently in different operating systems.
- AWT component is converted by the native code of the operating system.

LIMITATIONS OF AWT

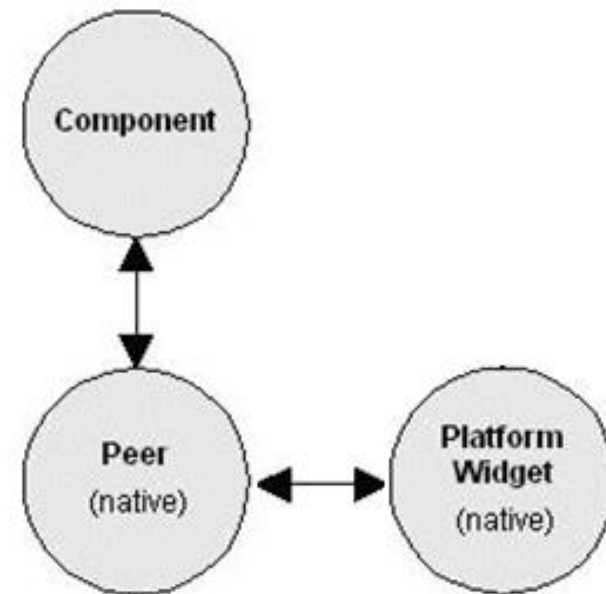
- **Lowest Common Denominator**
 - If not available natively on one Java platform, not available on any Java platform
- **Simple Component Set**
- **Components Peer-Based**
 - Platform controls component appearance
 - Inconsistencies in implementations
 - Interfacing to native platform error-prone

LOOK & FEEL (SWING | AWT)

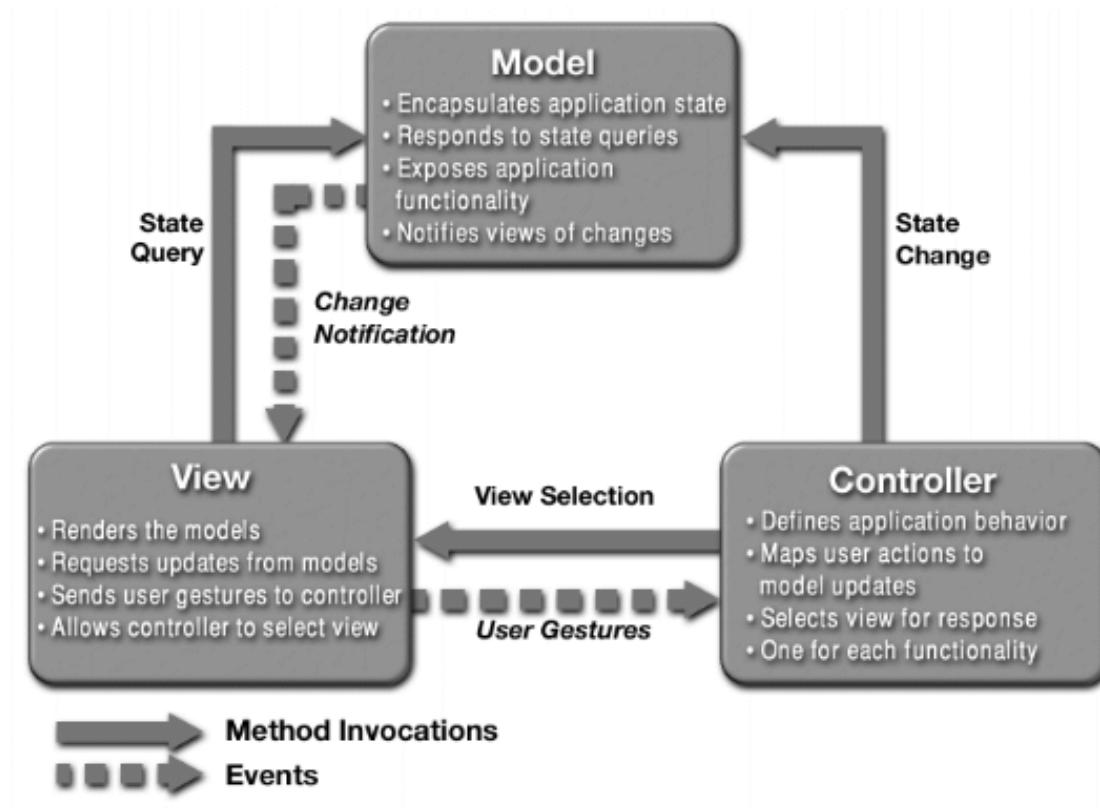
Swing Look & Feel



AWT Look & Feel



MVC ARCHITECTURE



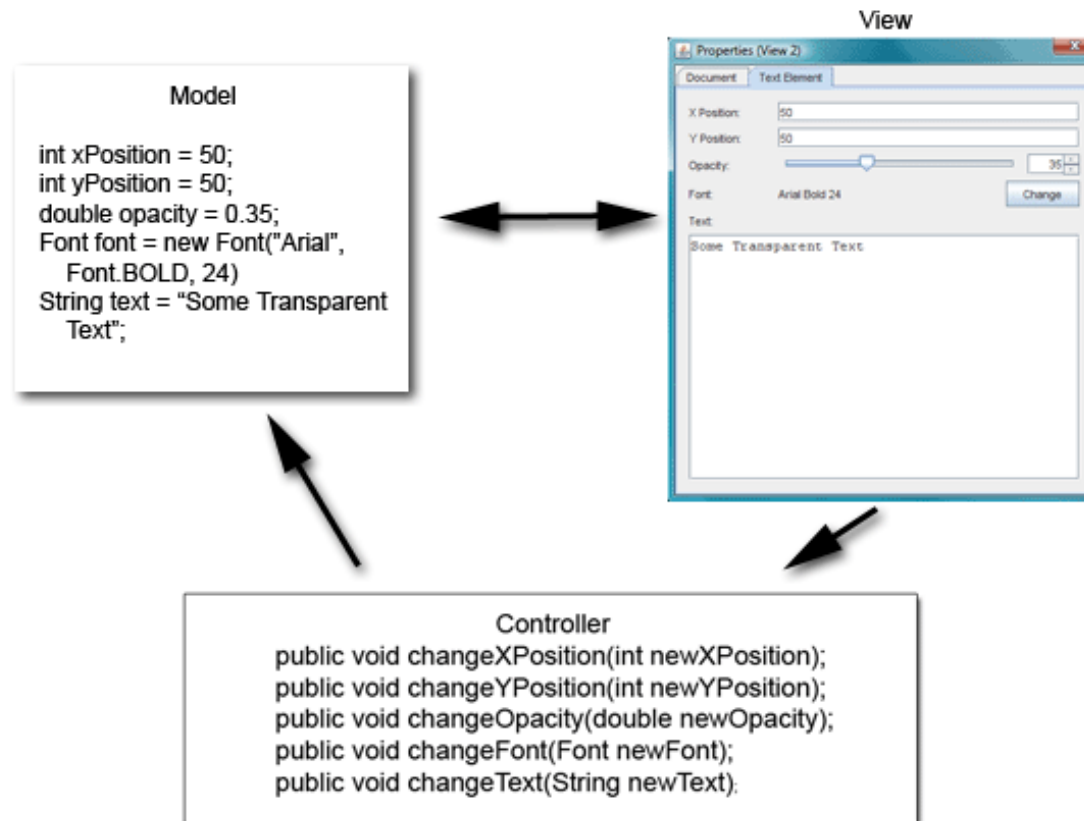
MODEL, VIEW

- **Model** - The model represents data and the rules that govern access to and updates of this data. In enterprise software, a model often serves as a software approximation of a real-world process.
- **View** - The view renders the contents of a model. It specifies exactly how the model data should be presented. If the model data changes, the view must update its presentation as needed. This can be achieved by using a push model, in which the view registers itself with the model for change notifications, or a pull model, in which the view is responsible for calling the model when it needs to retrieve the most current data.

CONTROLLER

- **Controller** - The controller translates the user's interactions with the view into actions that the model will perform. In a stand-alone GUI client, user interactions could be button clicks or menu selections, whereas in an enterprise web application, they appear as GET and POST HTTP requests. Depending on the context, a controller may also select a new view
- -- for example, a web page of results -- to present back to the user.

INTERACTION B/W MVC COMPONENTS



COMPONENTS

■ Container

■ JComponent

- AbstractButton
 - JButton
 - JMenuItem
 - JCheckBoxMenuItem
 - JMenu
 - JRadioButtonMenuItem
- JToggleButton
 - JCheckBox
 - JRadioButton

■ JComponent

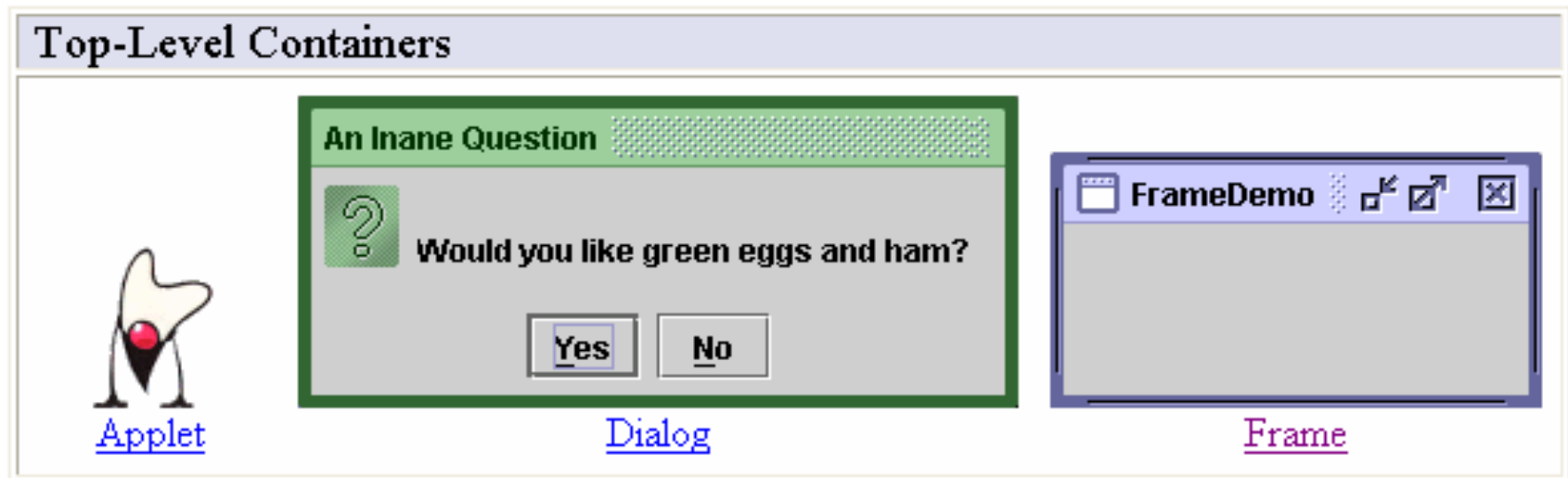
- JComboBox
- JLabel
- JList
- JMenuBar
- JPanel
- JPopupMenu
- JScrollBar
- JScrollPane

■ JComponent

- JTextComponent
 - JTextArea
 - JTextField
 - JPasswordField
 - JTextPane
 - JHTMLPane

CONTAINERS

- Top-Level Containers
- The components at the top of any Swing containment hierarchy



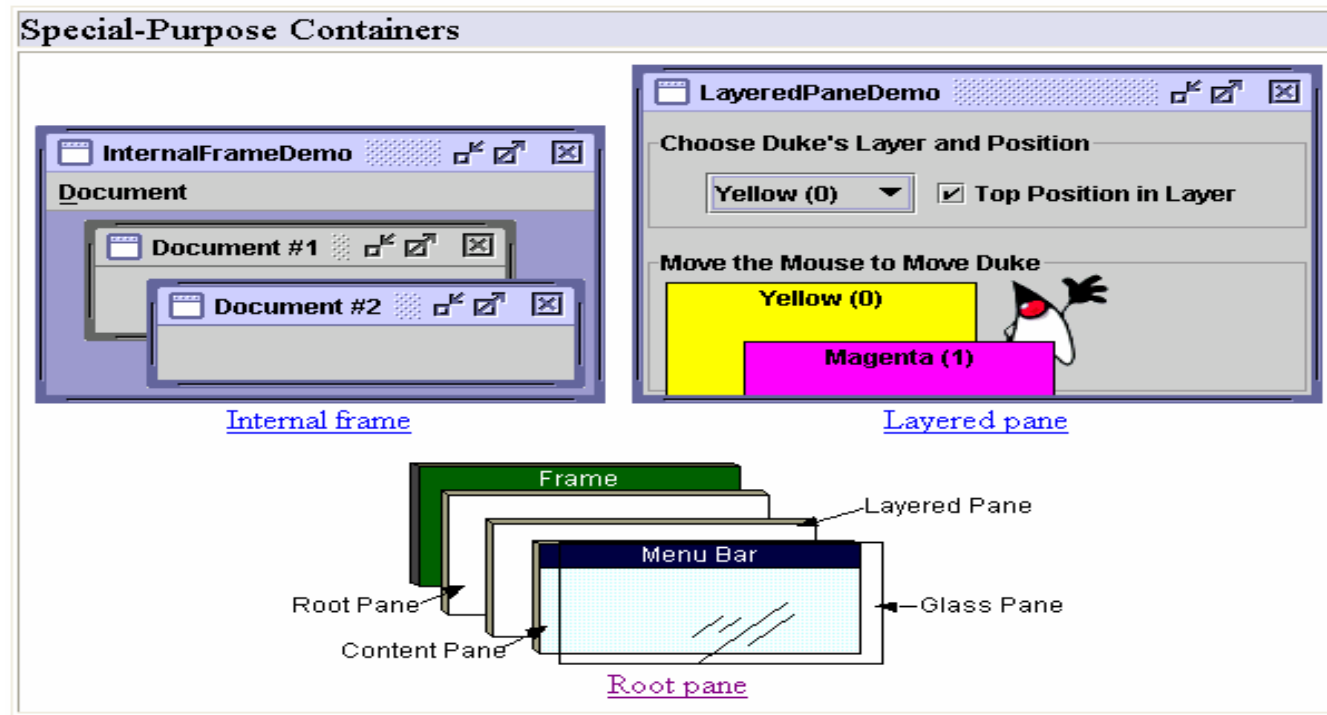
GENERAL PURPOSE CONTAINERS

- Intermediate containers that can be used under many different circumstances.



SPECIAL PURPOSE CONTAINER

- Intermediate containers that play specific roles in the UI.



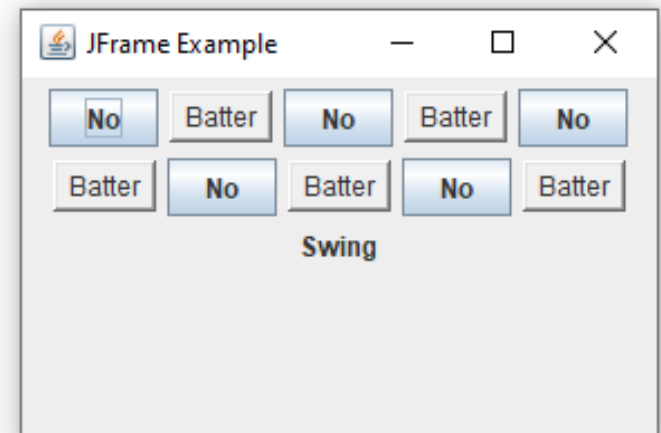
EXPLORING SWING - JAPPLET

- If using Swing components in an applet, subclass JApplet, not Applet
 - JApplet is a subclass of Applet
 - Sets up special internal component event handling, among other things
 - Can have a JMenuBar
 - Default LayoutManager is BorderLayout

JFRAME

```
import javax.swing.*;
import java.awt.*;

public class FrameTest {
    public static void main (String args[]) {
        JFrame f = new JFrame ("JFrame Example");
        Container c = f.getContentPane();
        c.setLayout (new FlowLayout());
        for (int i = 0; i < 5; i++) {
            c.add (new JButton ("No"));
            c.add (new Button ("Batter"));
        }
        c.add (new JLabel ("Swing"));
        f.setSize (300, 200);
        f.show();
    }
}
```



JCOMPONENT

- JComponent supports the following components.

JComboBox	JTextComponent
JLabel	JTextArea
JList	JTextField
JMenuBar	JPasswordField
JPanel	JTextPane
JPopupMenu	JHTMLPane
JScrollBar	
JScrollPane	

ICONS AND LABELS

- In Swing, icons are encapsulated by the ImageIcon class, which paints an icon from an image.
- constructors are:
 - ImageIcon(String filename)
 - ImageIcon(URL url)
- The ImageIcon class implements the Icon interface that declares the methods
 - int getIconHeight()
 - int getIconWidth()
 - void paintIcon(Component comp,Graphics g,int x, int y)

ICONS AND LABELS

- Swing labels are instances of the JLabel class, which extends JComponent. It can display text and/or an icon.
- Constructors are:
 - JLabel(Icon i) | Label(String s) | JLabel(String s, Icon i, int align)
- Here, s and i are the text and icon used for the label. The align argument is either LEFT, RIGHT, or CENTER. These constants are defined in the SwingConstants interface,
- Methods are:
 - Icon getIcon() | String getText() | void setIcon(Icon i)
 - void setText(String s)
- Here, i and s are the icon and text, respectively.

TEXT FIELDS

- The Swing text field is encapsulated by the `JTextComponent` class, which extends `JComponent`.
- It provides functionality that is common to Swing text components.
- One of its subclasses is `JTextField`, which allows you to edit one line of text.
- Constructors are:
 - `JTextField()` | `JTextField(int cols)` | `JTextField(String s, int cols)`
 - `JTextField(String s)`
- Here, `s` is the string to be presented, and `cols` is the number of columns in the text field.

BUTTONS

- Swing buttons provide features that are not found in the Button class defined by the AWT.
- Swing buttons are subclasses of the AbstractButton class, which extends JComponent.
- AbstractButton contains many methods that allow you to control the behavior of buttons, check boxes, and radio buttons.
- Methods are:
 - void setDisabledIcon(Icon di) | void setPressedIcon(Icon pi)
 - void setSelectedIcon(Icon si) | void setRolloverIcon(Icon ri)

BUTTONS CONTD...

- Here, di, pi, si, and ri are the icons to be used for these different conditions.
- The text associated with a button can be read and written via the following methods:
 - `String getText( )`
 - `void setText(String s)`
- Here, s is the text to be associated with the button.

JBUTTON

- The JButton class provides the functionality of a push button.
- JButton allows an icon, a string, or both to be associated with the push button.
- Some of its constructors are :
 - JButton(Icon i)
 - JButton(String s)
 - JButton(String s, Icon i)
- Here, s and i are the string and icon used for the button.

JCHECKBOX

- The JCheckBox class, which provides the functionality of a check box, is a concrete implementation of AbstractButton.
- Some of its constructors are shown here:
 - JCheckBox(Icon i) | JCheckBox(Icon i, boolean state)
 - JCheckBox(String s) | JCheckBox(String s, boolean state)
 - JCheckBox(String s, Icon i) | JCheckBox(String s, Icon i, boolean state)
- Here, i is the icon for the button. The text is specified by s. If state is true, the check box is initially selected. Otherwise, it is not.
- The state of the check box can be changed via the following method:
 - void setSelected(boolean state)
 - Here, state is true if the check box should be checked.

JCOMBOBOX

- Swing provides a combo box (a combination of a text field and a drop-down list) through the JComboBox class, which extends JComponent.
- A combo box normally displays one entry. However, it can also display a drop-down list that allows a user to select a different entry. You can also type your selection into the text field.
- Two of JComboBox's constructors are :
 - JComboBox() | JComboBox(Vector v)
 - Here, v is a vector that initializes the combo box.
- Items are added to the list of choices via the addItem() method, whose signature is:
 - void addItem(Object obj)
 - Here, obj is the object to be added to the combo box.

JRADIOBUTTON

- Radio buttons are supported by the `JRadioButton` class, which is a concrete implementation of `AbstractButton`.
- Some of its constructors are :
 - `JRadioButton(Icon i) | JRadioButton(Icon i, boolean state)`
 - `JRadioButton(String s) | JRadioButton(String s, boolean state)`
 - `JRadioButton(String s, Icon i) | JRadioButton(String s, Icon i, boolean state)`
- Here, `i` is the icon for the button. The text is specified by `s`. If state is true, the button is initially selected. Otherwise, it is not.
- Elements are then added to the button group via the following method:
 - `void add(AbstractButton ab)`
- Here, `ab` is a reference to the button to be added to the group.

JTABBEDPANE

- A tabbed pane is a component that appears as a group of folders in a file cabinet.
- Each folder has a title. When a user selects a folder, its contents become visible. Only one of the folders may be selected at a time.
- Tabbed panes are commonly used for setting configuration options. These are encapsulated by the JTabbedPane class, which extends JComponent. We will use its default constructor. Tabs are defined via the following method:
 - `void addTab(String str, Component comp)`
 - Here, `str` is the title for the tab, and `comp` is the component that should be added to the tab. Typically, a JPanel or a subclass of it is added.

JTABBEDPANE

- The general procedure to use a tabbed pane in an applet is outlined here:
 1. Create a JTabbedPane object.
 2. Call addTab() to add a tab to the pane. (The arguments to this method define the title of the tab and the component it contains.)
 3. Repeat step 2 for each tab.
 4. Add the tabbed pane to the content pane of the applet.

JSCROLLPANE

- A scroll pane is a component that presents a rectangular area in which a component may be viewed. Horizontal and/or vertical scroll bars may be provided if necessary.
- Scroll panes are implemented in Swing by the JScrollPane class, which extends JComponent. Some of its constructors are :
 - JScrollPane(Component comp)
 - JScrollPane(int vsb, int hsb)
 - JScrollPane(Component comp, int vsb, int hsb)
- Here, comp is the component to be added to the scroll pane. vsb and hsb are int constants that define when vertical and horizontal scroll bars for this scroll pane are shown.

JSCROLLPANE

- These constants are defined by the ScrollPaneConstants interface.
 - HORIZONTAL_SCROLLBAR_ALWAYS
 - HORIZONTAL_SCROLLBAR_AS_NEEDED
 - VERTICAL_SCROLLBAR_ALWAYS
 - VERTICAL_SCROLLBAR_AS_NEEDED
- Here are the steps to follow to use a scroll pane in an applet:
 - 1. Create a JComponent object.
 - 2. Create a JScrollPane object. (The arguments to the constructor specify the component and the policies for vertical and horizontal scroll bars.)
 - 3. Add the scroll pane to the content pane of the applet.

JTREE

- Data Model - `TreeModel`
 - default: `DefaultTreeModel`
 - `getChild`, `getChildCount`, `getIndexOfChild`, `getRoot`, `isLeaf`
- Selection Model - `TreeSelectionModel`
- View - `TreeCellRenderer`
 - `getTreeCellRendererComponent`
- Node - `DefaultMutableTreeNode`

JTABLE

- A table is a component that displays rows and columns of data. You can drag the cursor on column boundaries to resize columns. You can also drag a column to a new position.
- Tables are implemented by the `JTable` class, which extends `JComponent`.
- One of its constructors is :
 - `JTable(Object data[ ][ ], Object colHeads[ ])`
 - Here, `data` is a two-dimensional array of the information to be presented, and `colHeads` is a one-dimensional array with the column headings.

JTABLE

- Here are the steps for using a table in an applet:
 - 1. Create a JTable object.
 - 2. Create a JScrollPane object. (The arguments to the constructor specify the table and the policies for vertical and horizontal scroll bars.)
 - 3. Add the table to the scroll pane.
 - 4. Add the scroll pane to the content pane of the applet.

MENU BASICS

■ What is a Menu?

- A menu provides a space-saving way to let users choose from a list of options
- Typically appears as a horizontal bar at the top of a window
- Clicking a menu reveals a dropdown list of menu items
- Used in almost every desktop application (File, Edit, View, Help, etc.)

■ Why use menus?

- Organize commands logically
- Save screen space
- Provide keyboard shortcuts (mnemonics & accelerators)

MENU COMPONENT HIERARCHY

```
JComponent
|
+-- JMenuBar
|
+-- JMenuItem
|   |
|   +-- JMenu
|   +-- JCheckBoxMenuItem
|   +-- JRadioButtonMenuItem
|
+-- JPopupMenu
|
+-- JSeparator
```

JMENUBAR

- **JMenuBar Class**

- Container that holds a set of JMenu objects
- Usually placed at the top of a JFrame using setJMenuBar()

- **Key Methods:**

- add(JMenu m) – adds a menu to the menu bar
- remove(JMenu m) – removes a menu

- **Example:**

{ } Java

```
JFrame frame = new JFrame("Menu Example");  
JMenuBar menuBar = new JMenuBar();  
frame.setJMenuBar(menuBar);
```


JMENU

- **JMenu Class**

- Represents a pull-down menu component
- Deployed from a menu bar or as a submenu of another menu
- Contains JMenuItem objects

- **Key Methods:**

- add(JMenuItem item) – adds an item
- addSeparator() – adds a separator line
- setMnemonic(char c) – sets keyboard shortcut

- **Example:**

```
() Java
```

```
JMenu fileMenu = new JMenu("File");  
menuBar.add(fileMenu);  
fileMenu.setMnemonic('F');
```

JMENUITEM

- **JMenuItem Class**

- Represents a selectable choice in a menu
- Fires ActionEvent when clicked
- Can have icons, mnemonics, and accelerators

- **Example:**

() Java

```
JMenuItem openItem = new JMenuItem("Open");
openItem.setMnemonic('O');
openItem.setAccelerator(
    KeyStroke.getKeyStroke(KeyEvent.VK_O, InputEvent.CTRL_DOWN_MASK));
fileMenu.add(openItem);

openItem.addActionListener(e -> {
    System.out.println("Open clicked");
});
```

JCHECKBOXMENUITEM

- **JCheckBoxMenuItem Class**

- A menu item that can be checked or unchecked
- Maintains a boolean selected state
- Useful for toggle options (e.g., "Show Toolbar")

- **Key Methods:**

- `setState(boolean b)`
- `getState()`

Example:

`() Java`

```
JCheckBoxMenuItem boldItem =  
    new JCheckBoxMenuItem("Bold");  
formatMenu.add(boldItem);  
  
boldItem.addActionListener(e -> {  
    boolean isSelected = boldItem.getState();  
    System.out.println("Bold: " + isSelected);  
});
```

JRADIOBUTTONMENUITEM

- **JRadioButtonMenuItem Class**

- Menu item that works like a radio button
- Used in groups where only ONE item can be selected
- Must be grouped using ButtonGroup

- **Example:**

() Java

```
JRadioButtonMenuItem smallItem =  
    new JRadioButtonMenuItem("Small");  
JRadioButtonMenuItem largeItem =  
    new JRadioButtonMenuItem("Large");  
  
ButtonGroup group = new ButtonGroup();  
group.add(smallItem);  
group.add(largeItem);  
  
sizeMenu.add(smallItem);  
sizeMenu.add(largeItem);
```

JSEPARATOR

- **JSeparator Class**

- Provides a visual dividing line between groups of menu items
- Improves menu organization and readability
- **Note:** JMenu.addSeparator() is a shortcut for adding a JSeparator

- **Example:**

```
{ } Java
```

```
fileMenu.add(new JMenuItem("New"));  
fileMenu.add(new JMenuItem("Open"));  
fileMenu.addSeparator(); // Adds a line  
fileMenu.add(new JMenuItem("Exit"));
```

COMPLETE MENU BAR EXAMPLE

{ } Java

```
JFrame frame = new JFrame("Menu Demo");
JMenuBar menuBar = new JMenuBar();

JMenu fileMenu = new JMenu("File");
JMenuItem newItem = new JMenuItem("New");
JMenuItem openItem = new JMenuItem("Open");
JMenuItem exitItem = new JMenuItem("Exit");

fileMenu.add(newItem);
fileMenu.add(openItem);
fileMenu.addSeparator();
fileMenu.add(exitItem);

menuBar.add(fileMenu);
frame.setJMenuBar(menuBar);
frame.setSize(400, 300);
frame.setVisible(true);
```

CREATING A POPUP MENU

- **JPopupMenu Class**

- A menu that pops up at a specific location (right-click menu / context menu)
- Not attached to a menu bar
- Triggered programmatically, usually on mouse right-click

- **Basic Steps:**

- Create a JPopupMenu
- Add JMenuItem's to it
- Add a MouseListener to detect right-click
- Call show() to display the popup

POPUP MENU EXAMPLE

■ KeyPoints on Popup Menu

- Use `isPopupTrigger()` to detect the OS-specific right-click trigger
- Should check on BOTH `mousePressed` AND `mouseReleased` for cross-platform compatibility (Windows vs. Linux behave differently)
- `show(Component invoker, int x, int y)` displays the popup relative to a component

```
JPopupMenu popupMenu = new JPopupMenu();
JMenuItem cutItem = new JMenuItem("Cut");
JMenuItem copyItem = new JMenuItem("Copy");
JMenuItem pasteItem = new JMenuItem("Paste");
popupMenu.add(cutItem); popupMenu.add(copyItem);
popupMenu.add(pasteItem);
panel.addMouseListener(new MouseAdapter() {
    public void mousePressed(MouseEvent e) {
        checkPopup(e);
    }
    public void mouseReleased(MouseEvent e) {
        checkPopup(e);
    }
    private void checkPopup(MouseEvent e) {
        if (e.isPopupTrigger()) {
            popupMenu.show(e.getComponent(),
                           e.getX(), e.getY());
        }
    }
});
```


SUMMARY OF MENU CLASSES

Class	Purpose
JMenuBar	Container for menus, placed on JFrame
JMenu	Dropdown menu holding menu items
JMenuItem	Basic selectable menu option
JCheckBoxMenuItem	Toggleable checkbox menu item
JRadioButtonMenuItem	Single-select radio menu item (grouped)
JSeparator	Visual divider line in menus
JPopupMenu	Context menu shown on right-click